Table des matières

1 Introduction **3**

2 Environnement et outils utilisés **3**
2.1 Configuration logicielle 3
2.2 Principe de Hadoop MapReduce 3

3 Version 1 : Python simple **3**
3.1 Exécution 3

4 Version 2 : Pseudo-MapReduce Python **5**
4.1 Exécution 5

5 Version 3 : Hadoop MapReduce **6**
5.1 Installation et lancement de Hadoop 6
5.2 Étapes d'exécution du job 6
5.3 Résultat 6

6 Comparaison et analyse des performances **7**
6.1 Analyse 7

7 Conclusion **7**

1 Introduction

Le but de ce travail est de comparer trois approches pour le comptage du nombre d'occurrences de mots dans un texte :

— une version Python séquentielle simple ;
— une version pseudo-MapReduce en Python, reproduisant les étapes map, shuffle et reduce ;
— une version distribuée avec Hadoop MapReduce.

L'objectif est d'observer les différences entre les méthodes en termes de structure, outils utilisés et temps d'exécution.

2 Environnement et outils utilisés

2.1 Configuration logicielle

— **Système d'exploitation :** Manjaro Linux.
— **Langage :** Python 3.11.
— **Framework distribué :** Apache Hadoop 3.4.0.
— **Éditeur / Terminal :** Visual Studio Code.

2.2 Principe de Hadoop MapReduce

Hadoop MapReduce est un modèle de programmation distribué permettant de traiter de grands volumes de données. Il repose sur deux phases : Map (transformation des données en paires clé/valeur) et Reduce (agrégation des valeurs par clé). Le calcul est orchestré par YARN sur des données stockées dans HDFS.

3 Version 1 : Python simple

Le code suivant lit le fichier `mots.txt` et compte le nombre d'occurrences de chaque mot à l'aide d'un dictionnaire Python.

```python
def compter_occurrences(fichier):
    with open(fichier, 'r', encoding='utf-8') as f:
        mots = f.read().split()

    compteur = {}
    for mot in mots:
        compteur[mot] = compteur.get(mot, 0) + 1
    return compteur

if __name__ == "__main__":
    fichier = "mots.txt"
    resultats = compter_occurrences(fichier)
    for mot, nb in sorted(resultats.items(), key=lambda x: x[1], reverse=True):
        print(f"{mot} : {nb}")
```

Listing 1 – Code de la version Python simple

3.1 Exécution

Temps d'exécution : 0,012 s

```
$ time python wordcount.py
chat : 24
chien : 11
arbre : 7
fleur : 7
```

```
maison : 5
soleil : 5
montagne : 5
livre : 4
ciel : 4
mer : 3
lune : 3
route : 3
oiseau : 2
jardin : 1
voiture : 1

python wordcount.py  0,01s user 0,01s system 97% cpu 0,012 total
```

Listing 2 – Résultat de la version 1 sur le terminal.

4 Version 2 : Pseudo-MapReduce Python

Cette version reproduit les étapes du modèle MapReduce à l'aide de fonctions `mapper`, `shuffle` et `reducer`, exécutées localement.

```python
from collections import defaultdict

def mapper(ligne):
    mots = ligne.strip().split()
    return [(mot, 1) for mot in mots]

def shuffle(mapped_data):
    grouped = defaultdict(list)
    for mot, valeur in mapped_data:
        grouped[mot].append(valeur)
    return grouped

def reducer(grouped_data):
    reduced = {}
    for mot, valeurs in grouped_data.items():
        reduced[mot] = sum(valeurs)
    return reduced

if __name__ == "__main__":
    fichier = "mots.txt"
    mapped = []
    with open(fichier, "r", encoding="utf-8") as f:
        for ligne in f:
            mapped.extend(mapper(ligne))

    grouped = shuffle(mapped)
    reduced = reducer(grouped)

    for mot, count in sorted(reduced.items(), key=lambda x: x[1], reverse=True):
        print(f"{mot} : {count}")
```

Listing 3 – Code complet de la version pseudo-MapReduce

4.1 Exécution

Temps d'exécution : 0,017 s

```
$ time python mapreduce_wordcount.py
chat : 24
chien : 11
arbre : 7
fleur : 7
maison : 5
soleil : 5
montagne : 5
livre : 4
ciel : 4
mer : 3
lune : 3
route : 3
oiseau : 2
jardin : 1
voiture : 1

python mapreduce_wordcount.py 0,02s user 0,00s system 97% cpu 0,017 total
```

Listing 4 – Résultat de la version 2 sur le terminal.

5 Version 3 : Hadoop MapReduce

Cette version utilise la véritable infrastructure Hadoop pour exécuter le WordCount sur HDFS.

5.1 Installation et lancement de Hadoop

```
# Installation de Java et Hadoop
sudo pacman -S jdk-openjdk
wget .../hadoop-3.4.0.tar.gz
tar -xvzf hadoop-3.4.0.tar.gz && mv hadoop-3.4.0 ~/hadoop

# Configuration de l'environnement
echo "export HADOOP_HOME=$HOME/hadoop" >> ~/.bashrc
source ~/.bashrc

# D marrage du cluster
hdfs namenode -format
start-dfs.sh
start-yarn.sh
```

Listing 5 – Commandes d'installation et de démarrage

5.2 Étapes d'exécution du job

```
# 1. Creation du repertoire sur HDFS et copie des donnees
hdfs dfs -mkdir -p /user/imad/wordcount/input
hdfs dfs -put mots.txt /user/imad/wordcount/input/

# 2. Lancement du job MapReduce
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.4.0.jar \
wordcount /user/imad/wordcount/input /user/imad/wordcount/output

# 3. Visualisation du resultat
hdfs dfs -cat /user/imad/wordcount/output/part-r-00000
```

Listing 6 – Lancement du job MapReduce

5.3 Résultat

Durée totale : 12 secondes

Résumé de l'application YARN : application_1761131571423_0001

User :	imad
Name :	word count
Application Type :	MAPREDUCE
State :	FINISHED
Final Status :	SUCCEEDED
Started :	Wed Oct 22 13 :18 :30 +0000 2025
Finished :	Wed Oct 22 13 :18 :42 +0000 2025
Elapsed :	12sec
Tracking URL :	History

FIGURE 1 – Interface YARN montrant l'exécution du job WordCount (simulation textuelle).

6 Comparaison et analyse des performances

Méthode	Temps (s)	Remarques
Python simple	0,012	Lecture directe, très rapide sur petits fichiers.
Pseudo-MapReduce Python	0,017	Simulation des étapes Map/Reduce, surcharge faible.
Hadoop MapReduce	12	Latence au démarrage, mais adapté aux gros volumes.

Table 1 – Comparaison des trois méthodes de comptage de mots.

6.1 Analyse

Les versions Python sont beaucoup plus rapides sur de petits fichiers. Hadoop introduit une latence importante liée au démarrage des processus distribués. En revanche, Hadoop devient plus performant dès que la taille du jeu de données dépasse plusieurs gigaoctets, grâce à sa capacité de parallélisation.

7 Conclusion

Ce travail a permis d'illustrer le fonctionnement du modèle MapReduce à travers trois approches complémentaires : la simplicité du traitement local, la modularité du modèle simulé, et la puissance du traitement parallèle offert par Hadoop. Ces trois approches forment une progression naturelle : de la compréhension du concept à sa mise en œuvre réelle dans un environnement distribué.